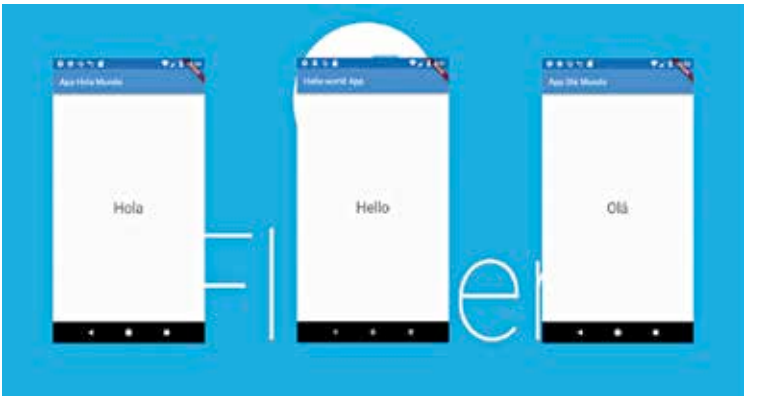
about 52 languages. For smooth functioning on iOS, the addition of the package called flutter_cupertino_localizations is also required.

Flutter also has provisions for languages with multiple variants. In this case, the Locale class is used to identify the language of the user. It is the mobile devices which support setting up a locale for all applications through systems settings. To this, the internationalised apps respond by displaying locale-specific values.



The Localisations widget, on the other hand, specifies localisations for its child and also the localised resources on which its child depends.

Flutter provides a WidgetsApp widget which in turns creates a localisation widget, rebuilding it if there is any change in the system locale.

Platform Integration

Writing the Code

Flutter makes use of a flexible system which allows one to call platform-specific APIs, Java or Kotlin code on Android and Objective-C or Swift code on iOS.

This platform-specific API support, instead of being dependant on code generation, relies on a flexible style which passes on messages.

What happens is that the Flutter part of the app sends messages to its host, which is the iOS or Android part of the app through a platform channel. The host receives the message through this platform channel and then calls on the platform-specific APIs using the native programming language and sends a report back to the Flutter part of the app-- the host.